

Algorithm Design and Analysis

Assignment 4

Deadline: May 31, 2026

1. (30 points) We are given a sequence of integers $a_1, a_2, \dots, a_n$, a lower bound, and an upper bound $1 \leq L \leq R \leq n$. An (L, R) -step subsequence is a subsequence $a_{i_1}, a_{i_2}, \dots, a_{i_\ell}$, such that $\forall 1 \leq j \leq \ell - 1, L \leq i_{j+1} - i_j \leq R$. The revenue of the subsequence is $\sum_{j=1}^{\ell} a_{i_j}$. Design a DP algorithm to output the maximum revenue we can get from a (L, R) -step subsequence.
 - (a) (10 points) Suppose $L = R = 1$. Design a DP algorithm in $O(n)$ to find the maximum $(1, 1)$ -step subsequence.
 - (b) (10 points) Design a DP algorithm in $O(n^2)$ to find the maximum (L, R) -step subsequence for any L and R .
 - (c) (10 points) Design a DP algorithm in $O(n)$ to find the maximum (L, R) -step subsequence for any L and R .
2. (30 points) **Optimal Indexing for A Dictionary:** Consider a dictionary with n different words $a_1, a_2, \dots, a_n$ sorted by the alphabetical order. We have already known the number of search times of each word a_i , which is represented by w_i . Suppose that the dictionary stores all words in a binary search tree T , i.e., each node's word is alphabetically larger than the words stored in its left subtree and smaller than the words stored in its right subtree. Then, to look up a word in the dictionary, we have to do $\ell_i(T)$ comparisons on the binary search tree, where $\ell_i(T)$ is exactly the level of the node that stores a_i (root has level 1). We evaluate the search tree by the total number of comparisons for searching the n words, i.e., $\sum_{i=1}^n w_i \ell_i(T)$. Design a DP algorithm to find the best binary search tree for the n words to minimize the total number of comparisons.

3. (40 points) **Precedence Constrained Unit Size Knapsack Problem** Let us recap the classic knapsack problem (unit size): We are given a knapsack of capacity W and n items, each item i has the same size $s_i = 1$ and is associated with its value v_i . The goal is to find a set of items S (each item can be selected at most once) with total size at most W (i.e., $\sum_{i \in S} s_i = |S| \leq W$) to maximize the total value (i.e., $\sum_{i \in S} v_i$). Now, we are given some additional precedence constraints among items; for example, we may have that item i can only be selected if we have selected item j . These constraints are presented by a *tree* $G = (V, E)$, which says that a vertex can only be selected if we have selected its parent, (i.e., we need to select all his ancestors). The task is also to maximize the total value we can get, but not violate the capacity constraint and any precedence constraints. Design a DP algorithm for it in $O(nW^2)$ time. **Tips.** You can try to improve the running time to $O(nW)$.
4. How long does it take you to finish the assignment (including thinking and discussing)? Give a score (1,2,3,4,5) to the difficulty. Do you have any collaborators? Write down their names here.